



TASK ASSIGNMENT MANUAL

Advanced Track — Assignment 2: Python Security Automation & Data Pipeline Engineering

Revision 2 — expanded scope: automated testing & scale requirements added

Track: AI, Automation & Security Engineering

1. Purpose & Overview

This manual assigns Assignment 2 of the Advanced Track (AI, Automation & Security Engineering) to all interns who have progressed beyond foundational GRC and scripting exercises. It evaluates the intern's ability to engineer production-quality Python automation for security data processing — including parsing diverse log formats, normalizing data into structured schemas, and implementing data quality validation. The assignment combines architectural reasoning (Theoretical) with applied software engineering (Technical 1 and Technical 2). Every intern assigned to this track is required to complete and submit all three tasks together as a single assignment.

2. Assignment Details

Company THE ARZENS	Program AI, Automation & Security Engineering — Advanced Track
Track AI, Automation & Security Engineering	Deadline Sunday, 26 July 2026, 11:59 PM (submitted via Google Classroom) — extended to accommodate the added testing & scale task
Issued By Malaika Saleem (GCR Lead) & Shayan Uddin (GCR Co-Lead)	Total Points 20 (5 points per task, 4 tasks)
Submission Format One combined PDF or ZIP: design memo (PDF/DOCX), log_parser.py, quality_validator.py, pytest test files, bulk-scale outputs, and PERFORMANCE.md	Submission Mode Individual — Google Classroom assignment upload only

3. Instructions

- Read all four tasks carefully before starting. Tasks 2, 3, and 4 are complementary — your validator in Task 3 should test data processed by your parser in Task 2, and your Task 4 stress test runs both of them together at scale.
- Tasks 2 and 3 each require a small pytest unit test suite alongside your script. This is a graded requirement, not optional polish — see each task's Testing Requirements subsection.
- Submit your work as a single upload on Google Classroom under this assignment post. Combine the design memo (as PDF or DOCX), the parser script (log_parser.py), the validator script (quality_validator.py), both pytest files, and your Task 4 outputs into one ZIP file, or upload them as separate attachments in the same submission.
- Name your files clearly, e.g., ArzensIntern_YourName_DesignMemo.pdf, ArzensIntern_YourName_log_parser.py, ArzensIntern_YourName_quality_validator.py.



- Late submissions will be marked as late by Google Classroom automatically; check the internship program's late-submission policy before assuming an extension applies.
- This is an individual assignment. You may discuss concepts with peers, but all submitted written analysis and code must be your own work.
- Where you use any AI assistance (e.g., for brainstorming, debugging, or drafting), disclose it briefly in a short “AI Assistance Note” at the end of your submission, stating the tool used and how it was used. Non-disclosure of AI assistance will be treated as an academic integrity violation, not the use of assistance itself.

4. Task 1 — Theoretical: Automation Design Memo (5 Points)

Write a 2-page design memo comparing quick scripts, production automation, and security engineering pipelines.

Requirements

- Length: approximately two pages (800–1,000 words), single-spaced, 11–12pt font.
- Structure the memo into three clear sections: (1) Quick Scripts — one-off, personal use automation; (2) Production Automation — team tools and shared utilities; (3) Security Engineering Pipelines — enterprise-grade, mission-critical systems.
- For each category, address:
 - Code structure and modularity expectations
 - Error handling requirements and failure modes
 - Logging, auditing, and observability standards
 - Testing expectations (unit tests, integration tests)
 - Documentation requirements and maintenance burden
- Include a brief conclusion recommending when to move from one category to the next based on operational impact and user base size.
- Support your analysis with realistic security operations examples (e.g., parsing a single log file vs. processing millions of daily events vs. real-time threat detection pipelines).

Evaluation Criteria (5 Points)

Evaluation Criteria	Points	What We're Looking For
Depth of comparison across all three categories	2.0	Thoroughness and accuracy of the Quick Scripts / Production Automation / Security Pipelines comparison
Realism of security operations examples	1.5	Concrete, realistic SecOps scenarios illustrating each category
Clarity of recommendations and decision criteria	1.0	Actionable, well-justified guidance on when to escalate categories
Professional writing and formatting	0.5	Grammar, structure, and presentation quality

5. Task 2 — Technical 1: Multi-Format Log Parser (5 Points)



Create a Python parser that converts three different security log formats into a normalized JSONL output.

Requirements

- Deliverable: a single `log_parser.py` file with accompanying sample input/output files. Your parser must handle three input formats:

Format	Style	Example
1 — Firewall Logs	CSV-style: timestamp,src_ip,dst_ip,port,action,bytes	2026-07-15T10:23:45Z,192.168.1.100,10.0.0.50,443,ALLOW,15234
2 — Authentication Logs	Space-delimited: date time user host status source_ip	2026-07-15 10:23:45 alice_web corporate-vpn SUCCESS 203.0.113.45
3 — DNS Logs	Key-value style: query_time=... client=... domain=... type=... response=...	query_time=2026-07-15T10:23:45Z client=192.168.1.50 domain=suspicious.example.com type=A response=NXDOMAIN

Output Schema (JSONL — one JSON object per line)

```
{
  "event_id": "auto-generated UUID (uuid4)",
  "timestamp": "ISO 8601 normalized (YYYY-MM-DDTHH:MM:SSZ)",
  "source_ip": "extracted IP or null",
  "target_ip": "extracted IP or null (firewall dst_ip)",
  "user": "extracted username or null",
  "action": "normalized verb (ALLOW/DENY/SUCCESS/FAILURE/QUERY/RESPONSE)",
  "status": "success/failure/unknown",
  "log_type": "firewall/auth/dns",
  "original_line": "preserved raw input for reference",
  "parsed_at": "ISO timestamp of parsing time"
}
```

Functional Requirements

- Handle at least 10 sample records per format (30 total minimum).
- Normalize timestamps to ISO 8601 UTC format.
- Validate IP addresses (IPv4 and IPv6 support preferred).
- Handle malformed lines gracefully (skip with warning, don't crash).
- Include a command-line interface (CLI) with an input file argument.
- Save output as a JSONL file (one JSON object per line).

Testing Requirements

- Include a pytest-based test suite (`test_log_parser.py`) with a minimum of 6 unit tests.
- Cover at least: one test per log format (3 total), one timestamp-normalization test, one IP-validation test, and one malformed-line/graceful-failure test.



- Tests must be runnable with a plain “pytest” command from the submission root and must pass on a clean checkout.

Deliverables

- log_parser.py — main Python script
- test_log_parser.py — pytest unit tests covering parsing, normalization, and error handling
- sample_firewall_logs.csv — 10+ firewall records
- sample_auth_logs.txt — 10+ authentication records
- sample_dns_logs.txt — 10+ DNS records
- output_normalized.jsonl — resulting normalized output
- README.md — brief usage instructions

Evaluation Criteria (5 Points)

Evaluation Criteria	Points	What We're Looking For
Correct parsing of all three formats	1.0	Firewall, authentication, and DNS logs all parse correctly
Accurate timestamp normalization	0.75	All timestamps normalized to ISO 8601 UTC
Proper IP validation and error handling	0.75	IPv4/IPv6 validated; malformed lines skipped, not crashed
Code structure, comments, and CLI interface	0.75	Modular, documented, runnable via command line
Unit test coverage (pytest)	1.0	6+ passing tests covering formats, normalization, and edge cases
Completeness of deliverables	0.75	All required files included and correctly named

6. Task 3 — Technical 2: Data Quality Validator (5 Points)

Build a Python validator that checks parsed events for quality issues and generates a compliance report.

Requirements

- Deliverable: a single quality_validator.py file that validates the output from Technical 1. Your validator must check for:

#	Check	Details
1	Missing Critical Fields	event_id, timestamp, source_ip, action, log_type must be present and non-null; report specific missing fields per record
2	Invalid IP Addresses	Validate IPv4 format (xxx.xxx.xxx.xxx, 0-255); flag private IP ranges in external-facing logs (10.x.x.x, 192.168.x.x, 172.16-31.x.x); report invalid formats
3	Timestamp Anomalies	Future timestamps; timestamps older than 1 year; unparseable timestamp formats



#	Check	Details
4	Duplicate Detection	Identify duplicate event_ids; report line numbers of duplicates
5	Suspicious Patterns	Impossible action/status combinations (e.g., SUCCESS with DENY); extreme byte counts (negative or >1TB); rapid sequential events (<1s apart)

Output Requirements

- Console report showing PASS/FAIL status for each check, a per-record error list, and summary statistics.
- CSV report with columns: line_number, check_type, severity (ERROR/WARNING/INFO), description, recommended_action.
- JSON summary with: total_records, valid_records, error_count, warning_count, by_check breakdown.

Sample Console Output

```
=====
DATA QUALITY VALIDATION REPORT
Generated: 2026-07-18T14:30:00Z
Input: output_normalized.jsonl
=====
```

```
Check 1: Missing Fields      PASS (30/30 records)
Check 2: IP Validation       WARNING (2 private IPs in external logs)
Check 3: Timestamp Anomalies PASS (0 issues)
Check 4: Duplicate Detection ERROR (3 duplicate event_ids: lines 15, 42, 89)
Check 5: Suspicious Patterns WARNING (1 impossible action)
```

SUMMARY:

```
Total Records: 30   Valid Records: 26   Errors: 3   Warnings: 3
```

Recommendations:

- Review duplicate event_ids for log source issues
- Investigate private IP logging in external-facing systems
- Check line 23 for impossible action/status combination

Testing Requirements

- Include a pytest-based test suite (test_quality_validator.py) with a minimum of 6 unit tests.
- Cover at least one test per check (5 total: missing fields, invalid IPs, timestamp anomalies, duplicates, suspicious patterns), plus one end-to-end test that runs the validator against a small sample JSONL file.
- Tests must be runnable with a plain “pytest” command from the submission root and must pass on a clean checkout.

Deliverables

- quality_validator.py — main validation script
- test_quality_validator.py — pytest unit tests covering all five checks plus an end-to-end run
- sample_validation_report.csv — example CSV output
- sample_validation_summary.json — example JSON summary



- README.md — brief usage instructions

Evaluation Criteria (5 Points)

Evaluation Criteria	Points	What We're Looking For
Completeness of all five validation checks	1.0	Each of the five checks implemented and functioning
Correct detection and reporting of issues	1.0	Accurate flagging of errors, warnings, and anomalies
Quality of output reports (readable, actionable)	0.75	Console, CSV, and JSON reports are clear and usable
Unit test coverage (pytest)	1.0	6+ passing tests covering all five checks plus an end-to-end run
Code clarity, comments, and error handling	0.75	Well-documented, robust against bad input
Inclusion of all required sample outputs	0.5	All sample deliverables present

7. Task 4 — Technical 3: Scale & Performance Stress Test (5 Points)

Stress-test your Task 2 parser and Task 3 validator against a larger synthetic dataset, and document their real-world performance characteristics. This task exists because production security pipelines process millions of events, not thirty — knowing how your code behaves under volume is what separates a script from a pipeline.

Requirements

- Generate a synthetic combined log dataset of at least 5,000 records, spread roughly evenly across the three formats from Task 2 (a small helper script, e.g. `generate_bulk_logs.py`, is the easiest way to do this — optional to submit but recommended).
- Run `log_parser.py` against the full dataset and confirm it completes without crashing, producing a valid `output_bulk.jsonl` file.
- Run `quality_validator.py` against `output_bulk.jsonl` and confirm it completes and produces a valid report.
- Measure and record: total records processed, total runtime (seconds), and throughput (records/second). Peak memory usage is a bonus if you have the tooling (e.g., `tracemalloc`) — a wall-clock timer alone is acceptable if not.
- Write a short `PERFORMANCE.md` (300–500 words) documenting your test setup, the measured numbers, at least one bottleneck you observed or expect at much larger scale (e.g., 1M+ records), and one concrete optimization you would apply (e.g., streaming line-by-line instead of loading the whole file into memory, batching writes, using generators, parallelizing per-format parsing).

Deliverables

- `output_bulk.jsonl` — parser output on the 5,000+ record dataset
- `bulk_validation_report.csv` (or `.json`) — validator output on that same dataset
- `PERFORMANCE.md` — performance notes and optimization analysis
- `generate_bulk_logs.py` — your data generator, if you wrote one (optional)



Evaluation Criteria (5 Points)

Evaluation Criteria	Points	What We're Looking For
Successful execution at scale	1.5	Parser and validator both complete on 5,000+ records without crashing
Performance measurement & documentation	1.5	Runtime and throughput accurately measured and reported
Bottleneck analysis & optimization recommendation	1.5	At least one realistic bottleneck identified with a concrete fix
Clarity of PERFORMANCE.md	0.5	Well-organized, readable, appropriately concise

8. Academic Integrity & Plagiarism Notice

All work submitted for this assignment must be your own original analysis and code. THE ARZENS Engineering Internship Program treats plagiarism, uncredited copying, and misrepresentation of AI-generated work as a serious violation of program conduct standards. The following applies to all three tasks:

- **Original work required** — Directly copying text from online sources, other interns' submissions, or AI tools without substantial original analysis and without disclosure is considered plagiarism.
- **No verbatim code** — Submitting code copied verbatim from public repositories or tutorials without understanding, adapting, and being able to explain it will be treated as a violation.
- **AI use must be disclosed** — You may use AI tools (e.g., Claude, ChatGPT, GitHub Copilot) to support brainstorming, debugging, or drafting, but you must disclose this use in a short "AI Assistance Note" at the end of your submission, and you remain fully responsible for the accuracy and originality of the final work.
- **Consequences** — Submissions found to be substantially copied or undisclosed AI output presented as original work will receive a zero for the affected task and may be escalated for review under the internship program's conduct policy.
- **When unsure, ask first** — If you are unsure whether a particular use of a source or tool is acceptable, ask your internship supervisor before the deadline rather than after submission.

9. Submission Checklist

☐ Design memo (Task 1) <i>PDF or DOCX, ~2 pages</i>	☐ log_parser.py (Task 2) <i>runnable Python script with CLI</i>	☐ Sample input files <i>firewall, auth, DNS — 10+ records each</i>
☐ output_normalized.jsonl <i>sample parser output</i>	☐ test_log_parser.py (Task 2) <i>pytest unit tests, 6+ passing</i>	☐ quality_validator.py (Task 3) <i>runnable Python script</i>
☐ sample_validation_report.csv <i>example validator output</i>	☐ sample_validation_summary.json <i>example JSON summary</i>	☐ test_quality_validator.py (Task 3) <i>pytest unit tests, 6+ passing</i>
☐ output_bulk.jsonl (Task 4) <i>parser output at 5,000+ record scale</i>	☐ bulk_validation_report (Task 4) <i>validator output at scale</i>	☐ PERFORMANCE.md (Task 4) <i>runtime, throughput, bottleneck</i>



THE ARZENS (TAS)

Engineering Internship Program — AI, Automation & Security Engineering Track

analysis

☐ **README files**

usage instructions for both scripts

☐ **AI Assistance Note**

included, if applicable

☐ **Files clearly named**

uploaded as one submission on Google Classroom

☐ **Submitted on time**

before Sunday 26 July 2026, 11:59 PM

10. Questions & Support

For clarifications about this assignment, reach out to the GRC Lead, Malaika Saleem, or the GRC Co-Lead, Shayan Uddin, before the submission deadline. Do not wait until the last day to raise questions.

The Arzens (TAS) — GRC Program — Advanced Track Task Assignment Manual